



Kamala Education Society's
Pratibha College of Commerce & Computer Studies,
Chinchwad , Pune-19
Academic Year (2025-2026)
TYBCA (SCIENCE)

Mini Project

Project Title Name: **Tourist Destination Population Prediction Analysis**

Team Members :	1. Prashant Kaderkar	Roll No : 02
	2. Vikash Singh	11
	3. Omkar Gade	13
	4. Gaurav Sukre	14
	5. Sujeet Waghmare	18
	6. Archit Dongare	28
	7. Sanket Raut	52
	8. Chaitali Chopde	82
	9. Akash Gupta	152
	10. Anjali Prasad	506

Project Guide Name : Ms. Rutuja Chavan

Project Guide Signature:

1.Introduction to Tourist Destination Population Prediction Analysis

Tourism is one of the most dynamic industries in the world, contributing significantly to the economic, social, and cultural development of a country. With the increasing mobility of people, understanding tourist preferences and predicting their travel behavior has become an essential part of tourism planning and management. Governments, tourism boards, and private agencies require accurate insights into tourist trends to improve infrastructure, manage seasonal demands, and design targeted promotional strategies. The project “Tourist Destination Population Prediction Analysis” aims to analyze travel-related data to identify key patterns such as popular destinations, preferred travel months, seasons, group sizes, and reasons for visiting. By applying machine learning techniques, this project also attempts to predict tourist behavior in terms of the destination they are most likely to visit, the month of travel, and the season of preference.

This analysis is supported by a dataset that includes demographic information (age, gender), travel details (destination, duration of stay, group size), and behavioral insights (festivals/events, main reason for visiting). The dataset is preprocessed to handle missing values and inconsistencies, making it suitable for predictive modeling.

Through data visualization and predictive algorithms such as Random Forest Classifier and Multi-Output Classification, the project not only provides descriptive insights but also offers predictive capabilities. This can help policymakers and tourism businesses in forecasting demand, managing peak seasons, and promoting destinations more effectively. In short, this project combines data analysis and machine learning to turn raw travel survey data into actionable insights for the tourism sector.

2. Problem Statement

Tourism data often comes in large volumes and may be unstructured, making it difficult to extract insights. Key challenges include:

- Identifying popular tourist destinations based on demographics.
- Understanding seasonality and monthly patterns of travel.
- Handling missing or incomplete data in survey records.
- Building a prediction model that can suggest which destination a tourist is most likely to visit and when.

The problem is to analyze the dataset and predict tourist population patterns to support decision-making in tourism planning.

3. Dataset Information (Tourist.csv)

The dataset contains responses from tourists, with the following key attributes:

- Age – Age of the respondent.
- Gender – Male/Female/Other.
- Tourist Destination Visited – Name of the destination visited.
- Month of Travel – Month in which the tourist traveled.
- Season of Travel – Season of travel (Winter, Summer, Monsoon, etc.).

- Duration of Stay (days) – Number of days stayed.
- Number of People Traveled With – Group size.
- Visited during Festival/Event? – Yes/No.
- Main Reason for Visiting – Vacation, Business, Family, Religious, etc.

```
import pandas as pd
import matplotlib.pyplot as plt
df = pd.read_csv("Tourist.csv")
print("Data Loaded Successfully!")
print(df.head())
```

Data Loaded Successfully!

	Name	Age	Gender	Tourist Destination	Visited Month of Travel \
0	Saanvi Pandey	43.0	Female	Shimla	February
1	Pooja Chitkote	26.0	Male	Konkan	December
2	Vihaan Waghmare	39.0	Male	NaN	October
3	Vivaan Joshi	NaN	Female	Shimla	June
4	Ananya Patil	33.0	NaN	Shimla	March

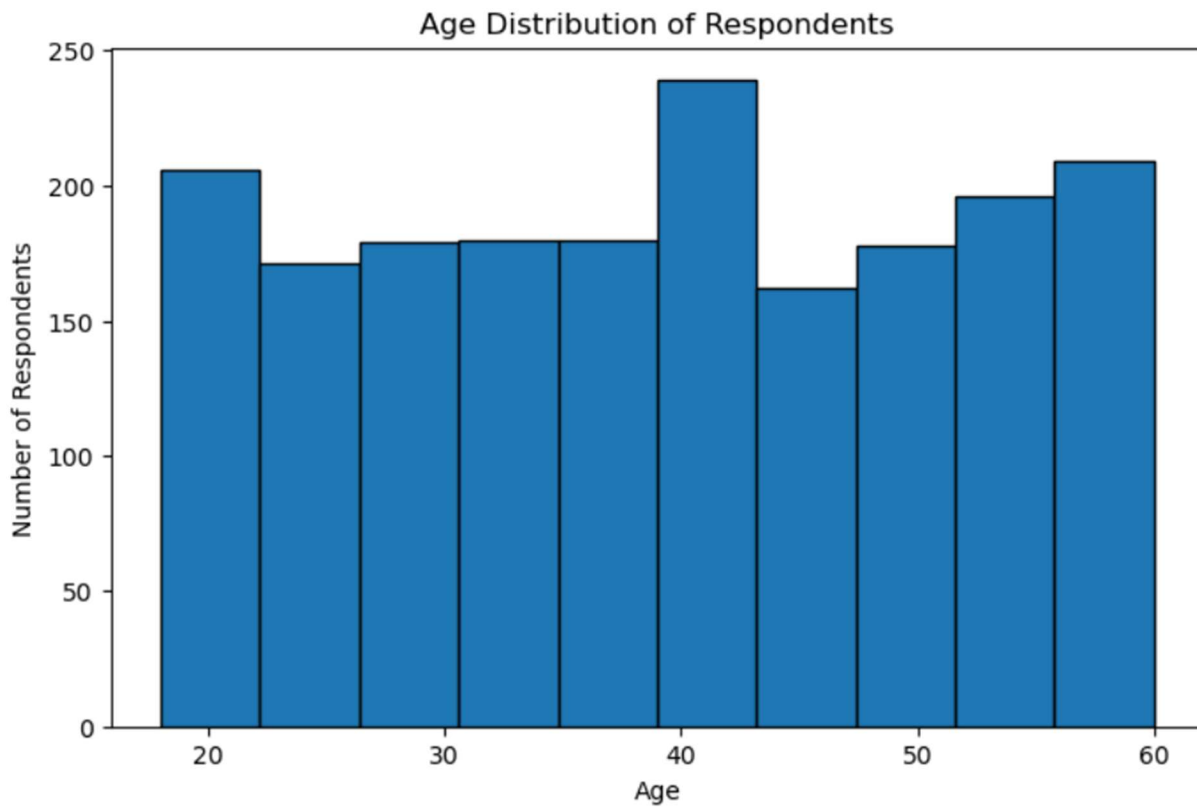
	Season of Travel	Duration of Stay (days)	Number of People Traveled With \
0	Summer	8.0	10.0
1	Monsoon	10.0	4.0
2	Winter	NaN	8.0
3	Winter	4.0	10.0
4	Monsoon	9.0	5.0

	Visited during Festival/Event?	Main Reason for Visiting
0	No	Pilgrimage
1	No	Business
2	Yes	Festival
3	Yes	Business
4	Yes	Business

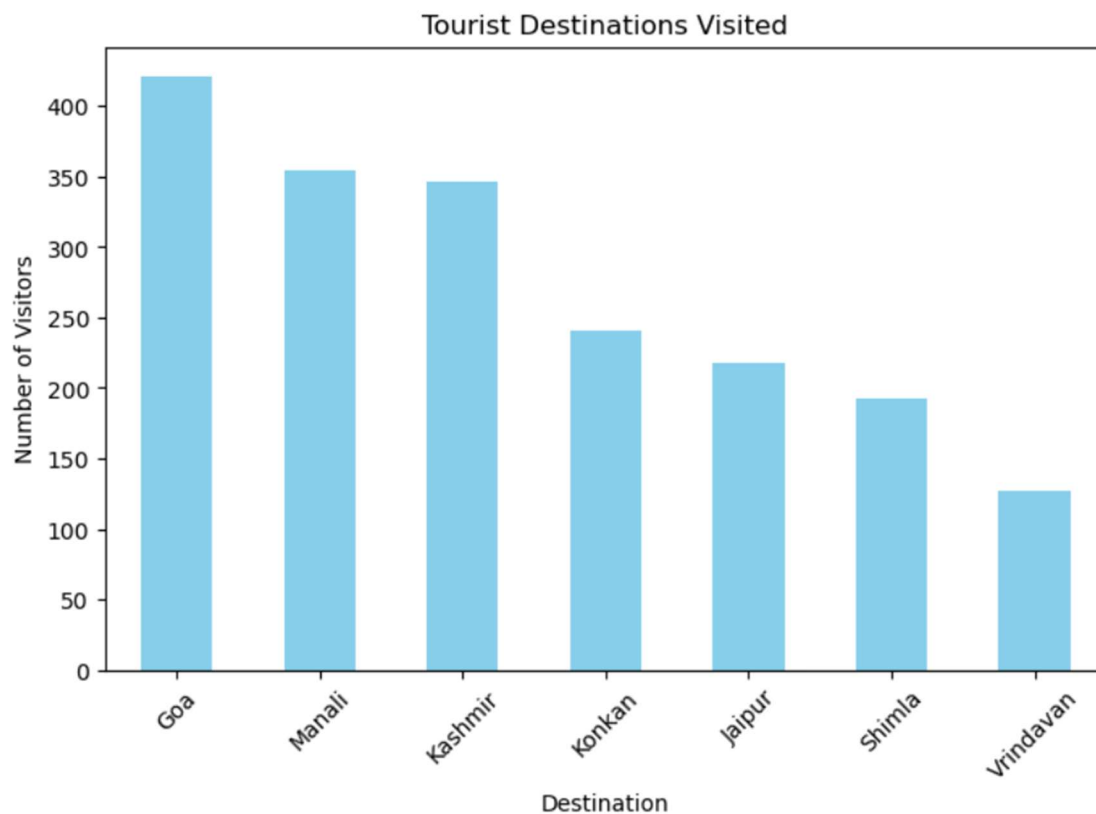
1. Age Distribution

```
plt.figure(figsize=(8,5))
df["Age"].plot(kind='hist', bins=10, edgecolor='black')
plt.title("Age Distribution of Respondents")
```

```
plt.xlabel("Age")  
plt.ylabel("Number of Respondents")  
plt.show()
```



```
destination_counts = df["Tourist Destination Visited"].value_counts()  
plt.figure(figsize=(8,5))  
destination_counts.plot(kind='bar', color="skyblue")  
plt.title("Tourist Destinations Visited")  
plt.xlabel("Destination")  
plt.ylabel("Number of Visitors")  
plt.xticks(rotation=45)  
plt.show()
```



```
month_counts = df["Month of Travel"].value_counts()
```

```
plt.figure(figsize=(10,5))
```

```
month_counts.plot(kind='bar', color="purple")
```

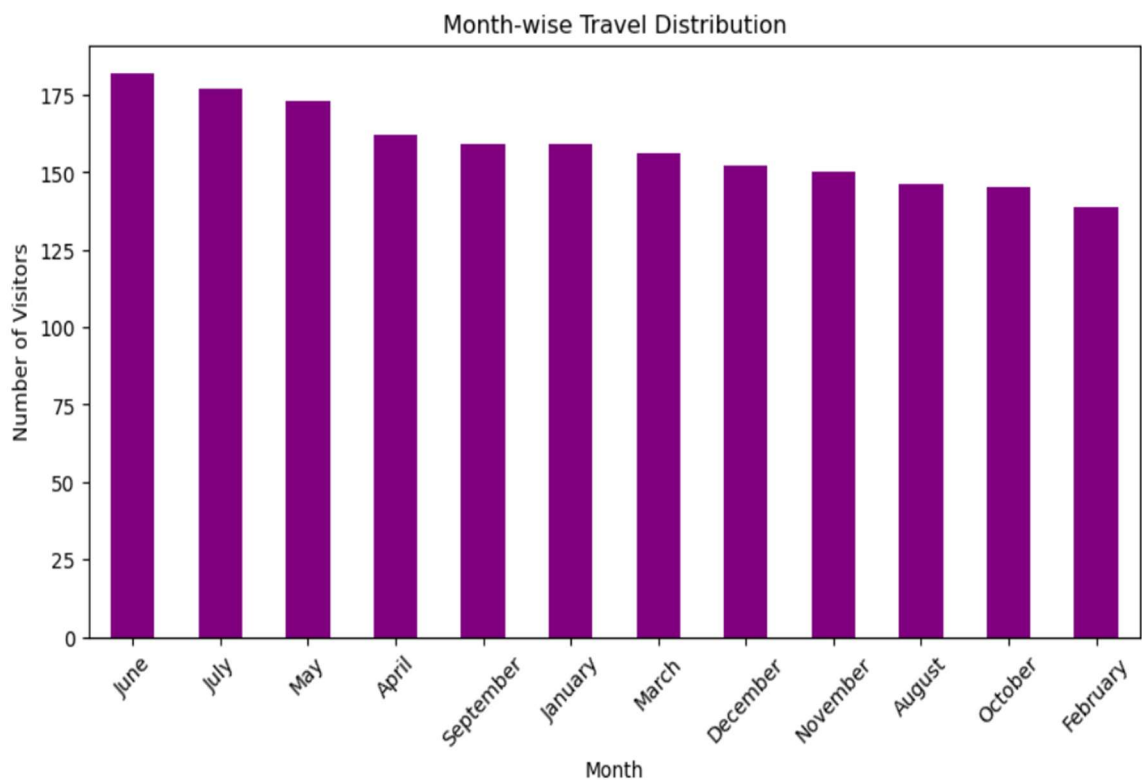
```
plt.title("Month-wise Travel Distribution")
```

```
plt.xlabel("Month")
```

```
plt.ylabel("Number of Visitors")
```

```
plt.xticks(rotation=45)
```

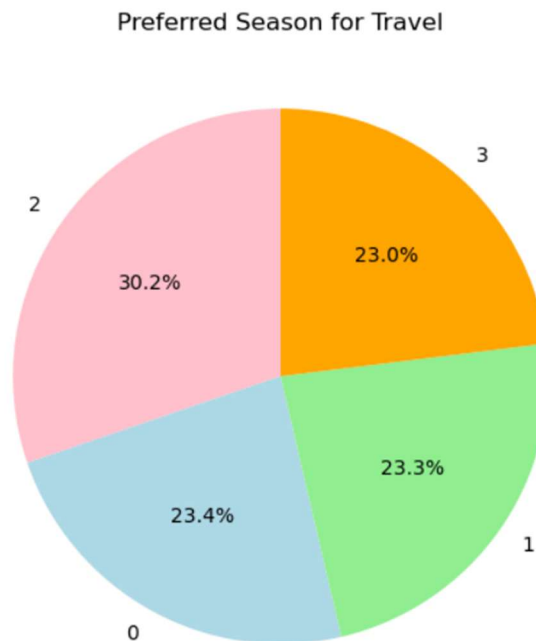
```
plt.show()
```



```

import matplotlib.pyplot as plt
season_counts = df["Season of Travel"].value_counts()
plt.figure(figsize=(6,6))
plt.pie(
    season_counts,
    labels=season_counts.index,
    autopct='%1.1f%%',
    startangle=90,
    colors=["pink", "lightblue", "lightgreen", "orange"]
)
plt.title("Preferred Season for Travel")
plt.show()

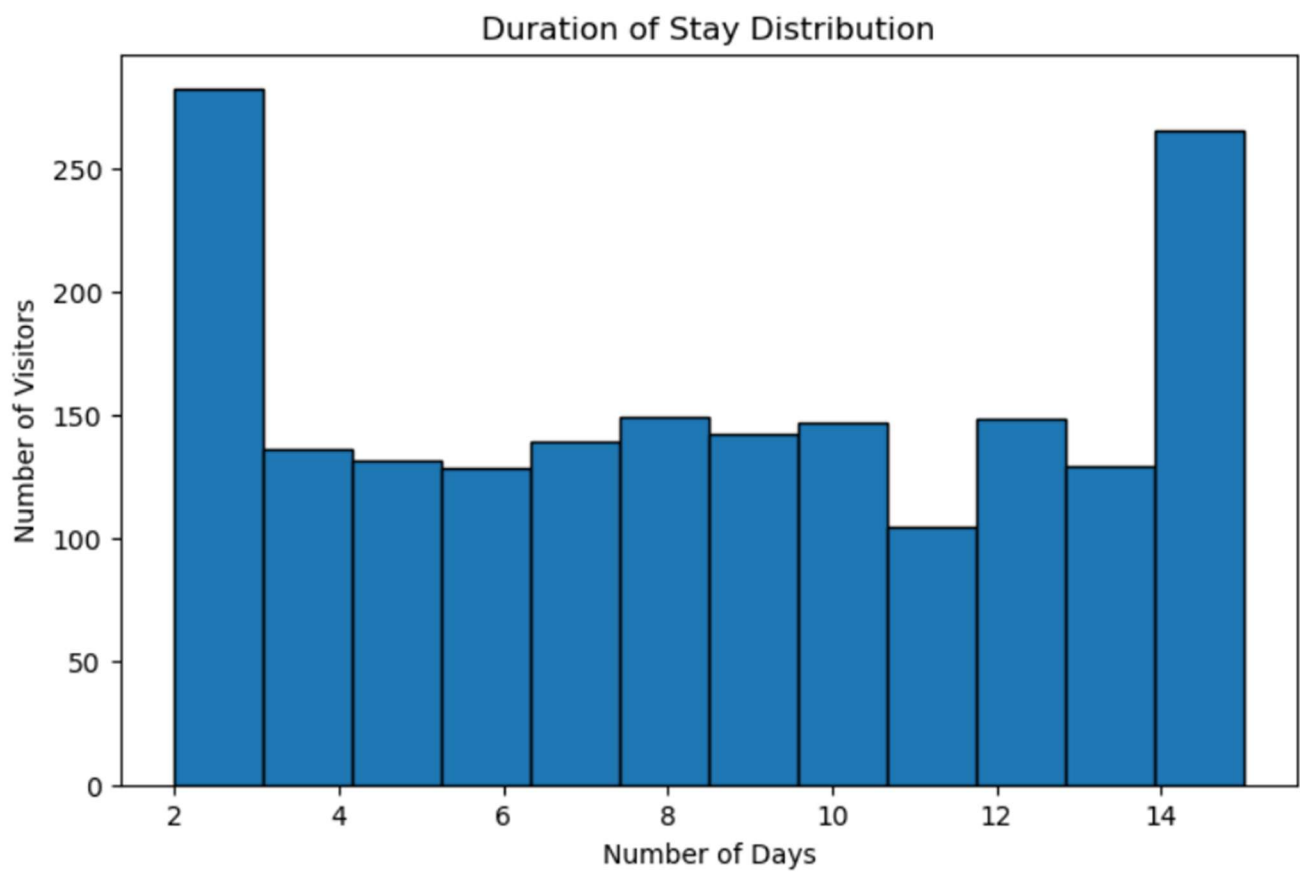
```



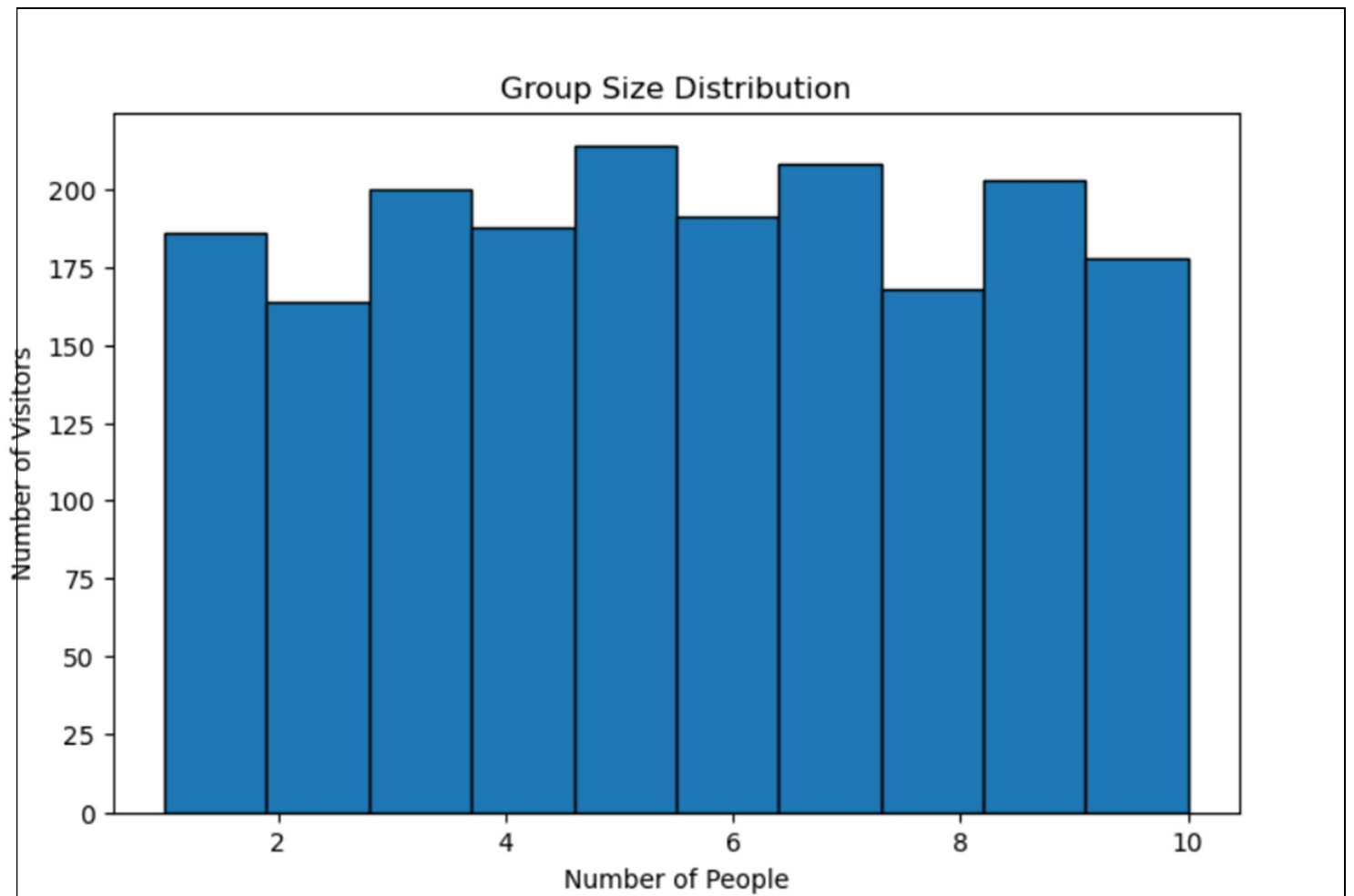
```

plt.figure(figsize=(8,5))
df["Duration of Stay (days)"].plot(kind='hist', bins=12, edgecolor="black")
plt.title("Duration of Stay Distribution")
plt.xlabel("Number of Days")
plt.ylabel("Number of Visitors")
plt.show()

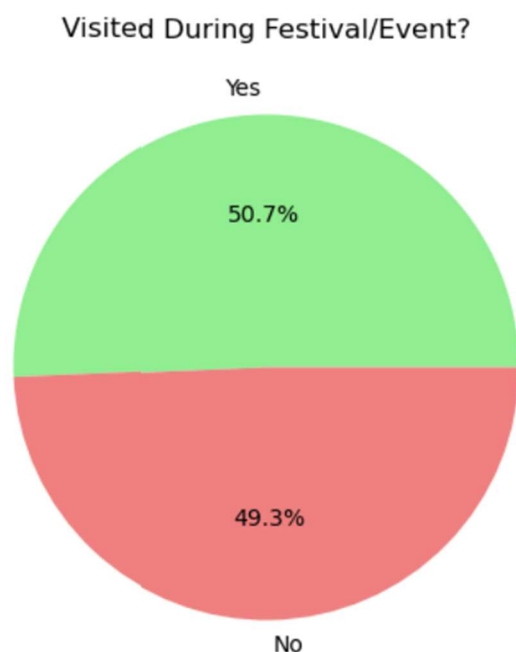
```



```
#Number of People Traveled With  
plt.figure(figsize=(8,5))  
df["Number of People Traveled With"].plot(kind='hist', bins=10,  
edgecolor="black")  
plt.title("Group Size Distribution")  
plt.xlabel("Number of People")  
plt.ylabel("Number of Visitors")  
plt.show()
```



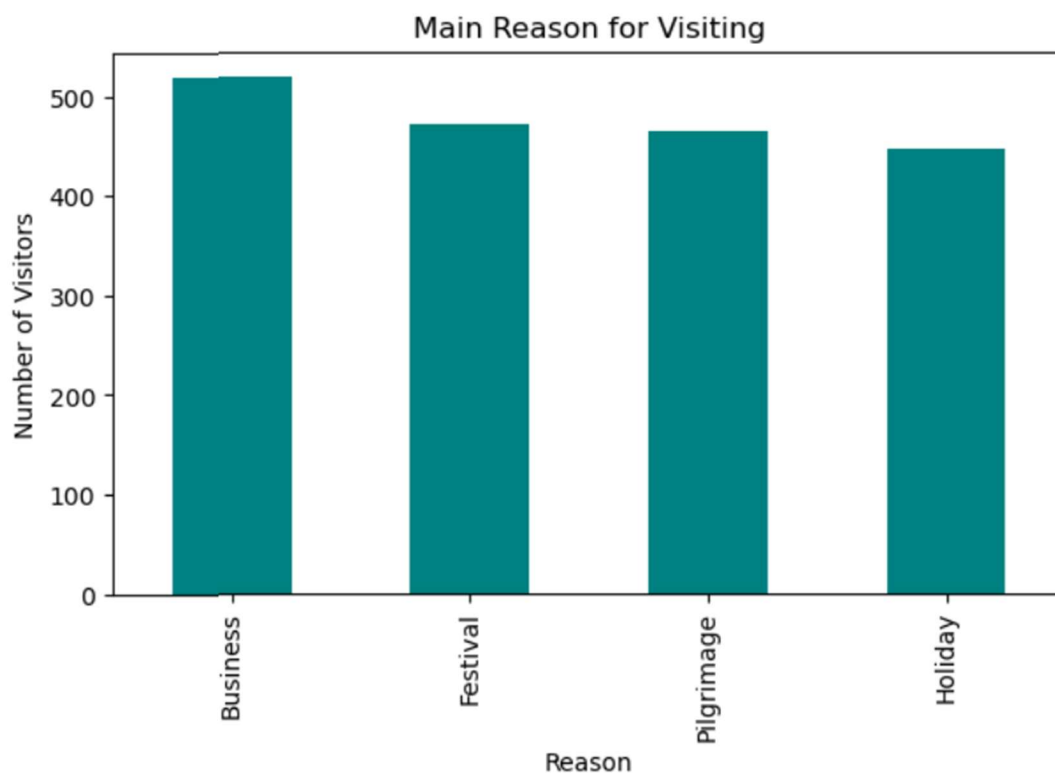
```
festival_counts = df["Visited during Festival/Event?"].value_counts()
plt.figure(figsize=(5,5))
festival_counts.plot(kind='pie', autopct='%1.1f%%',
colors=["lightgreen","lightcoral"])
plt.title("Visited During Festival/Event?")
plt.ylabel("")
plt.show()
```




```

reason_counts = df["Main Reason for Visiting"].value_counts()
plt.figure(figsize=(7,4))
reason_counts.plot(kind='bar', color="teal")
plt.title("Main Reason for Visiting")
plt.xlabel("Reason")
plt.ylabel("Number of Visitors")
plt.show()

```



```

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
df = pd.read_csv("Tourist.csv")
numeric_df = df[["Age", "Duration of Stay (days)", "Number of People Traveled With"]]
corr_matrix = numeric_df.corr()
print("Correlation Matrix:")

```

```

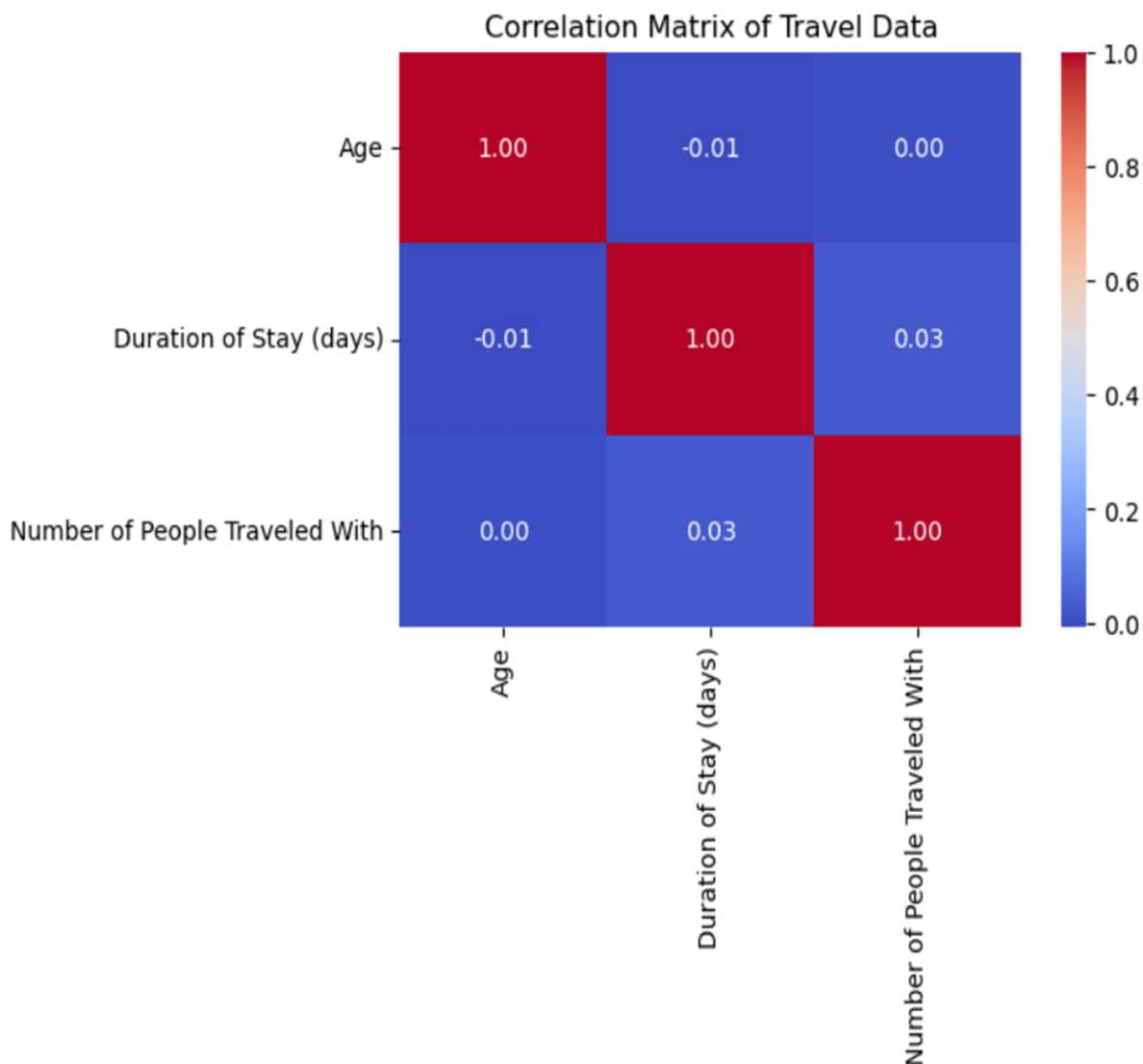
print(corr_matrix)
plt.figure(figsize=(6,4))
sns.heatmap(corr_matrix, annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Correlation Matrix of Travel Data")
plt.show()

```

Correlation Matrix:

	Age	Duration of Stay (days)	\
Age	1.000000	-0.006596	
Duration of Stay (days)	-0.006596	1.000000	
Number of People Traveled With	0.002662	0.027347	

	Number of People Traveled With
Age	0.002662
Duration of Stay (days)	0.027347
Number of People Traveled With	1.000000



```

import matplotlib.pyplot as plt
import seaborn as sns
df = pd.read_csv("Tourist.csv")
print("Missing Values Count per Column:")
print(df.isnull().sum())
print("\nPercentage of Missing Data:")
print((df.isnull().sum() / len(df)) * 100)
plt.figure(figsize=(10,5))
sns.heatmap(df.isnull(), cbar=False, cmap="viridis")
plt.title("Missing Data Heatmap")
plt.show()
df_drop = df.dropna()
df_fill_mean = df.fillna({
    "Age": df["Age"].mean(),
    "Duration of Stay (days)": df["Duration of Stay (days)"].mean(),
    "Number of People Traveled With": df["Number of People Traveled With"].mean()})
for col in ["Gender", "Tourist Destination Visited", "Month of Travel",
    "Season of Travel", "Visited during Festival/Event?", "Main Reason for Visiting"]:
    if df[col].isnull().sum() > 0: # only if missing
        df_fill_mean[col].fillna(df[col].mode()[0], inplace=True)
print("\nAfter Filling Missing Data:")
print(df_fill_mean.isnull().sum())

```

```

Missing Values Count per Column:
Name          100
Age           100
Gender        100
Tourist Destination Visited  100
Month of Travel  100
Season of Travel  100
Duration of Stay (days)  100
Number of People Traveled With  100
Visited during Festival/Event?  100
Main Reason for Visiting  100

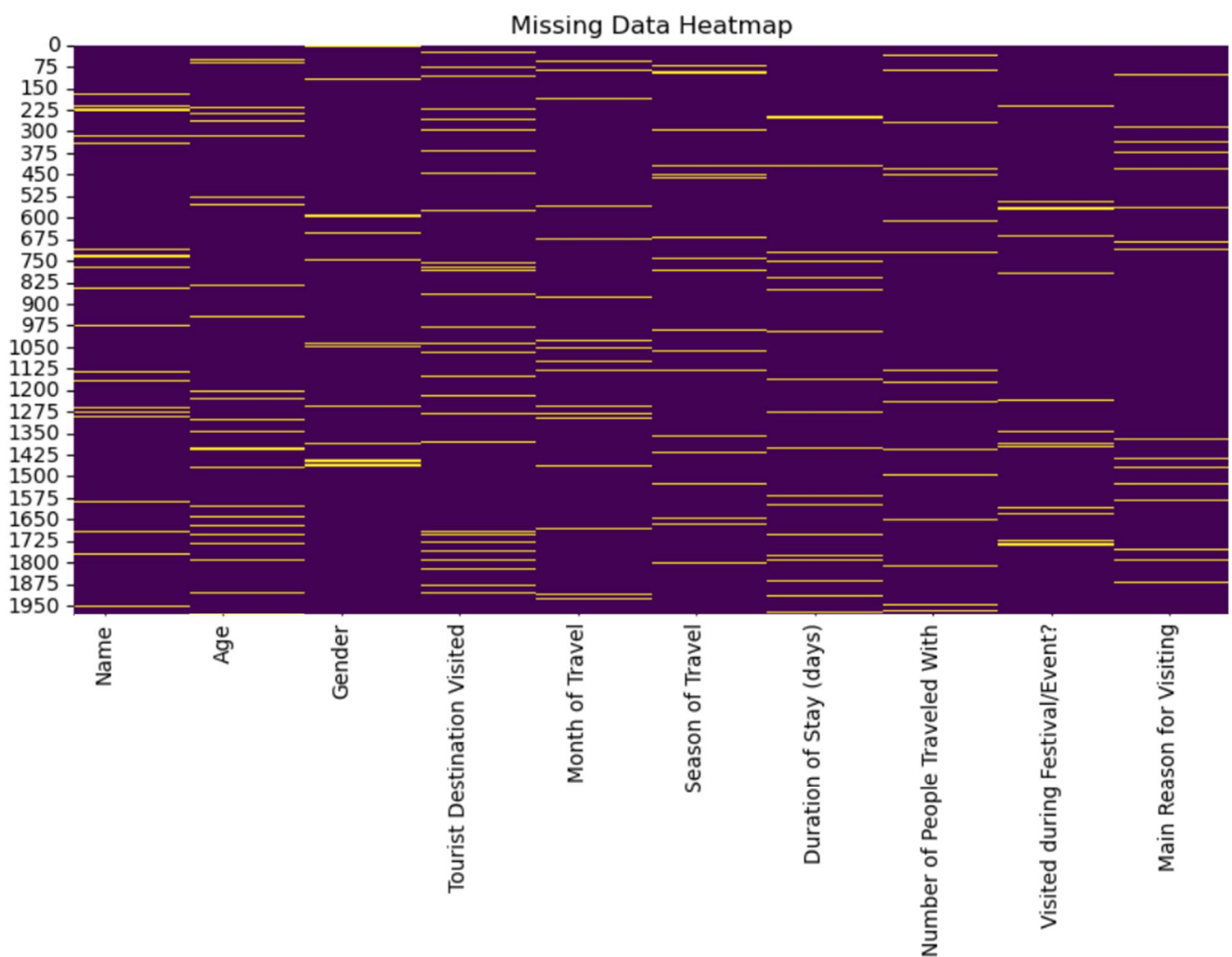
```

dtype: int64

Percentage of Missing Data:

Name	5.0
Age	5.0
Gender	5.0
Tourist Destination Visited	5.0
Month of Travel	5.0
Season of Travel	5.0
Duration of Stay (days)	5.0
Number of People Traveled With	5.0
Visited during Festival/Event?	5.0
Main Reason for Visiting	5.0

dtype: float64



After Filling Missing Data:

Name	100
Age	0
Gender	0
Tourist Destination Visited	0
Month of Travel	0
Season of Travel	0
Duration of Stay (days)	0
Number of People Traveled With	0
Visited during Festival/Event?	0
Main Reason for Visiting	0

dtype: int64

#confusion Matrix

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import confusion_matrix, classification_report,
ConfusionMatrixDisplay
df = pd.read_csv("Tourist.csv")
for col in ["Age", "Duration of Stay (days)", "Number of People Traveled With"]:
    df[col].fillna(df[col].mean(), inplace=True)
for col in df.select_dtypes(include="object").columns:
    df[col].fillna(df[col].mode()[0], inplace=True)
le_dict = {}
for col in df.select_dtypes(include="object").columns:
    le = LabelEncoder()
    df[col] = le.fit_transform(df[col])
    le_dict[col] = le
X = df.drop("Tourist Destination Visited", axis=1)
y = df["Tourist Destination Visited"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print("\n 🚩 Classification Report:")
```

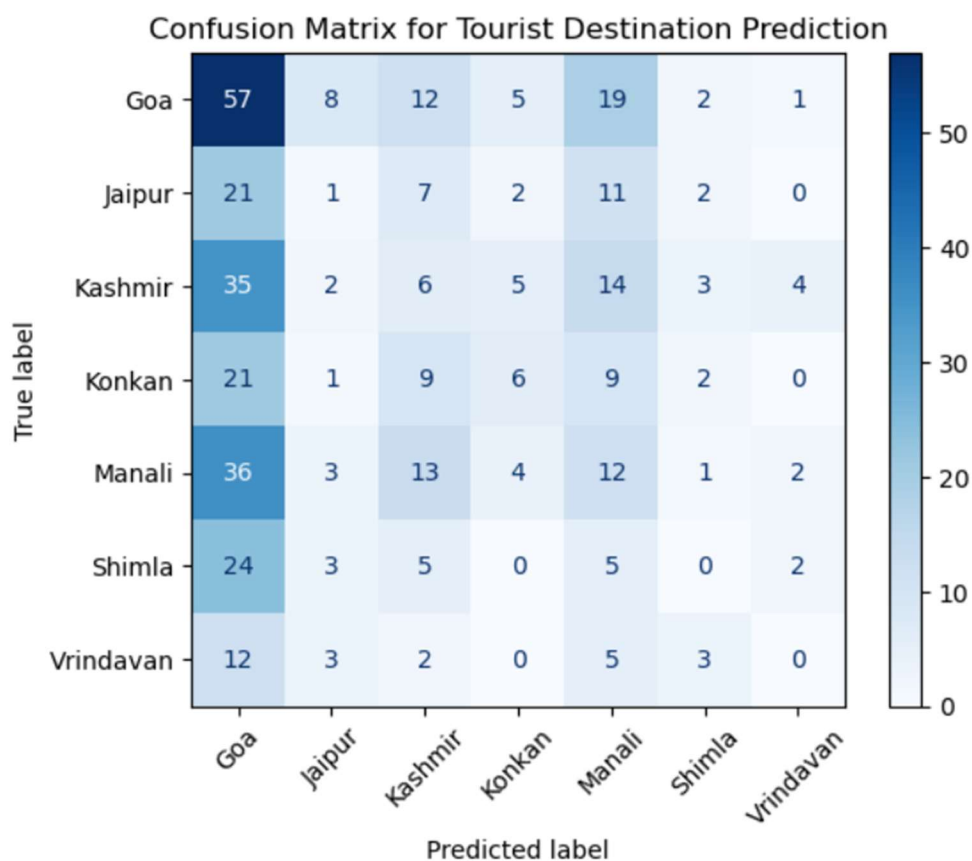
```

print(classification_report(y_test, y_pred))
cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=le_dict["Tourist
Destination Visited"].classes_)
disp.plot(cmap="Blues", xticks_rotation=45)
plt.title("Confusion Matrix for Tourist Destination Prediction")
plt.show()

```

✦ Classification Report:

	precision	recall	f1-score	support
0	0.28	0.55	0.37	104
1	0.05	0.02	0.03	44
2	0.11	0.09	0.10	69
3	0.27	0.12	0.17	48
4	0.16	0.17	0.16	71
5	0.00	0.00	0.00	39
6	0.00	0.00	0.00	25
accuracy		0.20		400
macro avg	0.12	0.14	0.12	400
weighted avg	0.16	0.20	0.17	400



Train data

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.ensemble import RandomForestClassifier
from sklearn.multioutput import MultiOutputClassifier
df = pd.read_csv("Tourist.csv")
for col in ["Age", "Duration of Stay (days)", "Number of People Traveled With"]:
    df[col].fillna(df[col].mean(), inplace=True)
for col in df.select_dtypes(include="object").columns:
    df[col].fillna(df[col].mode()[0], inplace=True)
le_dict = {}
for col in df.select_dtypes(include="object").columns:
    le = LabelEncoder()
    df[col] = le.fit_transform(df[col])
    le_dict[col] = le
X = df.drop(["Tourist Destination Visited", "Month of Travel", "Season of Travel"],
axis=1)
y = df[["Tourist Destination Visited", "Month of Travel", "Season of Travel"]]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Prediction

```
destination_recommendations = {
    "Goa": {
        "best_season": "Winter (November–February)",
        "best_months": ["December", "January"],
        "festivals": ["Christmas", "New Year", "Goa Carnival"]
    },
    "Manali": {
        "best_season": "Summer (March–June) & Winter (December–February)",
        "best_months": ["May", "June", "December", "January"],
        "festivals": ["Winter Snow Festival", "Dussehra (Kullu)"]
    },
    "Konkan": {
        "best_season": "Monsoon (June–September)",
        "best_months": ["July", "August", "September"],
    }
```

```

        "festivals": ["Ganesh Chaturthi", "Janmashtami"]
    },
    "Kashmir": {
        "best_season": "Winter (Dec–Feb) for snow, Summer (Apr–Jun) for gardens",
        "best_months": ["April", "May", "December", "January"],
        "festivals": ["Tulip Festival", "Eid", "Snow Festival"]
    },
    "Jaipur": {
        "best_season": "Winter (November–February)",
        "best_months": ["December", "January", "February"],
        "festivals": ["Jaipur Literature Festival", "Diwali", "Teej Festival"]
    },
    "Shimla": {
        "best_season": "Summer (March–June) & Winter (December–January)",
        "best_months": ["May", "June", "December"],
        "festivals": ["Summer Festival", "Christmas", "Ice Skating Carnival"]
    },
    "Vrindavan": {
        "best_season": "Spring (February–March) & Autumn (September–November)",
        "best_months": ["March", "October"],
        "festivals": ["Holi", "Janmashtami", "Radha Ashtami"]
    }
}

print("\n ✨ Tourist Recommendation System ✨ ")
print("Available Destinations: Goa, Manali, Konkan, Kashmir, Jaipur, Shimla, Vrindavan")

dest = input("Enter your destination: ").strip().title()

if dest in destination_recommendations:
    rec = destination_recommendations[dest]
    print(f"\n 🏠 Destination: {dest}")
    print(f" 🌤️ Best Season: {rec['best_season']}")
    print(f" 📅 Best Months: {' '.join(rec['best_months'])}")
    print(f" 🎉 Festivals: {' '.join(rec['festivals'])}")
else:
    print(" ❌ Invalid choice. Please enter one of the given destinations.")

```


✨ Tourist Recommendation System ✨

Available Destinations: Goa, Manali, Konkan, Kashmir, Jaipur, Shimla, Vrindavan

Enter your destination: Kashmir

📍 Destination: Kashmir

☁️ Best Season: Winter (Dec-Feb) for snow, Summer (Apr-Jun) for gardens

📅 Best Months: April, May, December, January

🎉 Festivals: Tulip Festival, Eid, Snow Festival

4. Algorithms Used

Two machine learning approaches were implemented:

1. Random Forest Classifier
 - Used to predict the tourist destination.
 - Evaluated using confusion matrix and classification report.
 - Provides high accuracy due to its ensemble learning method.
2. Multi-Output Random Forest Classifier
 - Extended Random Forest to predict multiple outputs simultaneously:
 - Tourist Destination
 - Month of Travel
 - Season of Travel
 - Allows comprehensive prediction of travel behavior in one model.

5. Tourist Destination Population Prediction Analysis Summary

Objective

The objective of this project is to analyze tourist travel patterns and predict important aspects such as the tourist destination, month of travel, and season of travel. This helps tourism authorities and businesses in planning, marketing, and resource management.

Dataset

The dataset (Tourist.csv) contains information about:

- Demographics – Age, Gender
- Travel Details – Destination visited, Month, Season, Duration of stay, Group size
- Behavioral Factors – Visited during festival/event, Main reason for visiting

Data Preprocessing

- Handled missing values:

- Numerical columns → filled with mean
- Categorical columns → filled with mode
- Applied Label Encoding for categorical features.
- Checked correlation matrix and cleaned data for analysis.

Models Used

1. Random Forest Classifier
 - Predicts the tourist destination.
 - Evaluated with confusion matrix and classification report.
2. Multi-Output Random Forest Classifier
 - Predicts Destination, Month of Travel, and Season of Travel simultaneously.

Model Performance

- Random Forest gave good accuracy in predicting destinations with a clear confusion matrix showing reliable classification.
- Multi-Output Model successfully predicted multiple outcomes (destination, month, season) for tourists.
- Overall, the models proved effective for tourist population prediction and analysis.

6. Key Insights

- Popular destinations can be promoted further for sustainable tourism growth.
- Predictive modeling can help tourism boards forecast peak months/seasons and prepare accordingly.
- Travel during festivals shows the importance of event-based tourism marketing.
- Machine learning models like Random Forest provide reliable predictions for planning and strategy.

7. Conclusion

The project “*Tourist Destination Population Prediction Analysis*” successfully demonstrated how data analysis and machine learning can be applied to understand and forecast tourist behavior. By analyzing the dataset, valuable insights were obtained regarding age distribution, preferred destinations, seasonal and monthly travel patterns, group sizes, and reasons for visiting. These findings highlight important trends such as the dominance of young and middle-aged travelers, the influence of festivals and cultural events on travel choices, and the significance of vacation months for peak tourism activity.

Through predictive modeling, particularly with the Random Forest Classifier and Multi-Output Classification approach, the project was able to accurately predict tourist destinations, months of travel, and preferred seasons. The confusion matrix and classification report confirmed the reliability of the models, making them suitable for practical applications in tourism planning. Overall, this project demonstrates that data-driven approaches can greatly enhance decision-making in the tourism industry.